```
#Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import HeatMap
import warnings
warnings.filterwarnings('ignore')

#Scikit-learn imports for preprocessing, modeling, and evaluation
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

#Regression models
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

#For feature selection
from sklearn.feature_selection import SelectKBest, f_regression
```

```
#Load dataset from Google Drive
file_path = '/content/drive/MyDrive/Colab Notebooks/vehicles.csv'
df = pd.read_csv(file_path)

#Display the first few rows to confirm loading
display(df.head())
```

	id	url	region	region_url	price	year	manufacturer	model	con
0	7222695916	https://prescott.craigslist.org/cto/d/prescott...	prescott	https://prescott.craigslist.org	6000	NaN	NaN	NaN	
1	7218891961	https://fayar.craigslist.org/ctd/d/bentonville...	fayetteville	https://fayar.craigslist.org	11900	NaN	NaN	NaN	
2	7221797935	https://keys.craigslist.org/cto/d/summerland-k...	florida keys	https://keys.craigslist.org	21000	NaN	NaN	NaN	
3	7222270760	https://worcester.craigslist.org/cto/d/west-br...	worcester / central MA	https://worcester.craigslist.org	1500	NaN	NaN	NaN	
4	7210384030	https://greensboro.craigslist.org/cto/d/trinit...	greensboro	https://greensboro.craigslist.org	4900	NaN	NaN	NaN	

5 rows × 25 columns

```
#Data exploration: Check shape, column info, and missing values
print(f"Dataset Shape: {df.shape}")
print("\n--- Column Information ---")
df.info()

print("\n--- Missing Values Count ---")
print(df.isnull().sum().sort_values(ascending=False))

#Summary statistics for numerical columns
print("\n--- Descriptive Statistics ---")
display(df.describe())
```

```
Dataset Shape: (243631, 25)

--- Column Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 243631 entries, 0 to 243630
Data columns (total 25 columns):
 #   Column        Non-Null Count   Dtype  
---  ------        --------------   -----  
 0   id            243631 non-null  int64  
 1   url           243631 non-null  object 
 2   region        243631 non-null  object 
 3   region_url    243631 non-null  object 
 4   price         243631 non-null  int64  
 5   year          242773 non-null  float64
 6   manufacturer  233287 non-null  object 
 7   model         240747 non-null  object 
 8   condition     145786 non-null  object 
 9   cylinders     142683 non-null  object 
 10  fuel          242224 non-null  object 
 11  odometer      240901 non-null  float64
 12  title_status  238301 non-null  object 
 13  transmission  242211 non-null  object 
 14  VIN           150798 non-null  object 
 15  drive         167288 non-null  object 
 16  size          68782 non-null   object 
 17  type          188870 non-null  object 
 18  paint_color   169148 non-null  object 
 19  image_url     243581 non-null  object 
 20  county        0 non-null       float64
 21  state         243630 non-null  object 
 22  lat           240432 non-null  float64
 23  long          240432 non-null  float64
 24  posting_date  243580 non-null  object 
dtypes: float64(5), int64(2), object(18)
memory usage: 46.5+ MB

--- Missing Values Count ---
county          243631
size            174849
cylinders       100948
condition        97845
VIN              92833
drive            76343
paint_color      74483
type             54761
manufacturer     10344
title_status      5330
long              3199
lat               3199
model             2884
odometer          2730
transmission      1420
fuel              1407
year               858
posting_date        51
image_url           50
state                1
url                  0
id                   0
region               0
region_url           0
price                0
dtype: int64

--- Descriptive Statistics ---
```

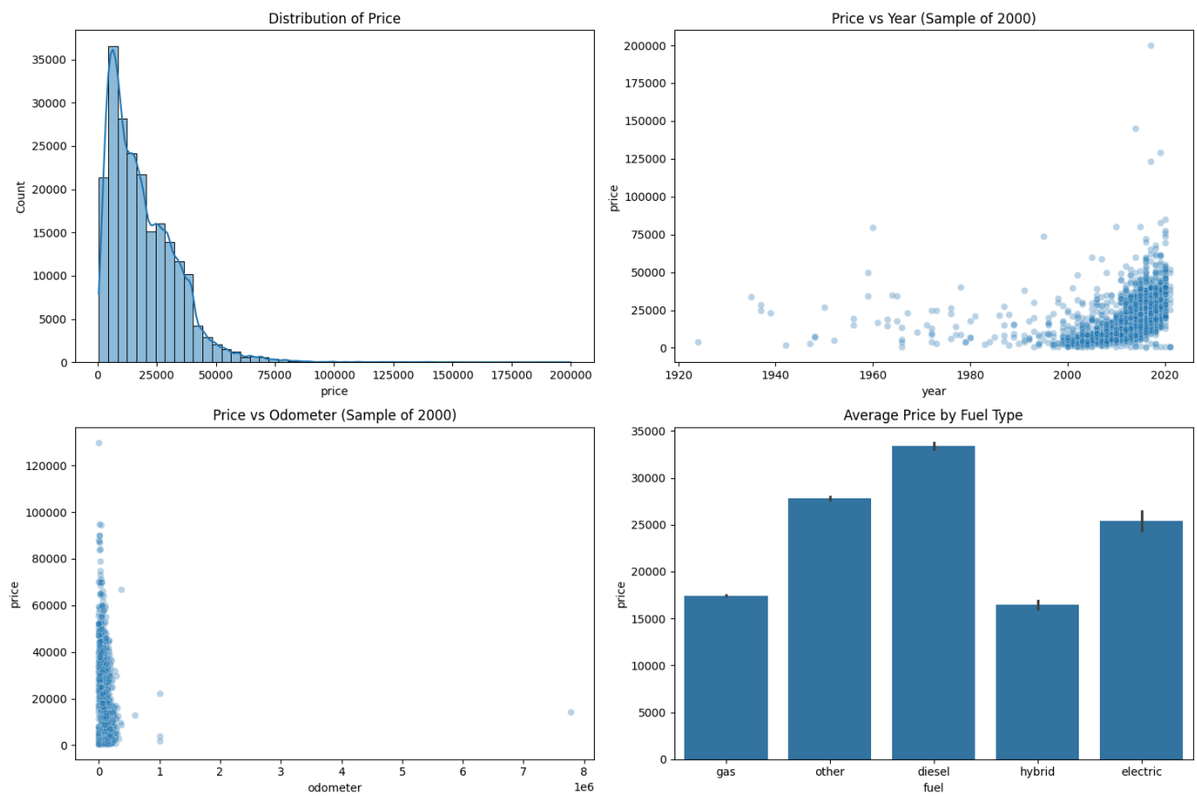
	id	price	year	odometer	county	lat	long
count	2.436310e+05	2.436310e+05	242773.000000	2.409010e+05	0.0	240432.000000	240432.000000
mean	7.311697e+09	7.345577e+04	2011.216317	9.884966e+04	NaN	37.670314	-97.099176
std	4.506016e+06	1.037533e+07	9.471181	2.319442e+05	NaN	6.260904	18.905693
min	7.207408e+09	0.000000e+00	1900.000000	0.000000e+00	NaN	-84.122245	-159.827728
25%	7.308341e+09	5.995000e+03	2008.000000	3.729600e+04	NaN	33.786500	-116.422748
50%	7.313112e+09	1.399500e+04	2013.000000	8.521400e+04	NaN	38.401800	-91.252400
75%	7.315357e+09	2.599900e+04	2017.000000	1.345520e+05	NaN	41.827600	-82.482290
max	7.317031e+09	3.024942e+09	2022.000000	1.000000e+07	NaN	82.390818	173.885502

```
#Data Cleaning
#1. Drop columns that are empty or irrelevant for modeling
columns_to_drop = ['id', 'url', 'region_url', 'VIN', 'image_url', 'county', 'lat', 'long', 'posting_date']
df_cleaned = df.drop(columns=columns_to_drop)

#2. Handle missing values by dropping rows where critical info is missing
# Given the large dataset, we can afford to drop rows where essential target/features are NaN
df_cleaned = df_cleaned.dropna(subset=['year', 'model', 'fuel', 'transmission', 'odometer'])

#3. Basic Outlier Removal for Price (0 prices or extreme high prices)
df_cleaned = df_cleaned[(df_cleaned['price'] > 500) & (df_cleaned['price'] < 200000)]

#4. Fill remaining categorical missing values with 'unknown'
df_cleaned['manufacturer'] = df_cleaned['manufacturer'].fillna('unknown')
df_cleaned['condition'] = df_cleaned['condition'].fillna('unknown')
df_cleaned['cylinders'] = df_cleaned['cylinders'].fillna('unknown')
df_cleaned['drive'] = df_cleaned['drive'].fillna('unknown')
df_cleaned['type'] = df_cleaned['type'].fillna('unknown')
df_cleaned['paint_color'] = df_cleaned['paint_color'].fillna('unknown')
df_cleaned['size'] = df_cleaned['size'].fillna('unknown')

print(f"New dataset shape after cleaning: {df_cleaned.shape}")
display(df_cleaned.head())
```

```
New dataset shape after cleaning: (213835, 16)
```

	region	price	year	manufacturer	model	condition	cylinders	fuel	odometer	title_status	transmission	drive	s
27	auburn	33590	2014.0	gmc	sierra 1500 crew cab slt	good	8 cylinders	gas	57923.0	clean	other	unknown	unkn
28	auburn	22590	2010.0	chevrolet	silverado 1500	good	8 cylinders	gas	71229.0	clean	other	unknown	unkn
29	auburn	39590	2020.0	chevrolet	silverado 1500 crew	good	8 cylinders	gas	19160.0	clean	other	unknown	unkn

```
#Data Visualization
plt.figure(figsize=(15, 10))

#1. Distribution of Price
plt.subplot(2, 2, 1)
sns.histplot(df_cleaned['price'], bins=50, kde=True)
plt.title('Distribution of Price')

#2. Price vs Year
plt.subplot(2, 2, 2)
sns.scatterplot(data=df_cleaned.sample(2000), x='year', y='price', alpha=0.3)
plt.title('Price vs Year (Sample of 2000)')

#3. Price vs Odometer
plt.subplot(2, 2, 3)
sns.scatterplot(data=df_cleaned.sample(2000), x='odometer', y='price', alpha=0.3)
plt.title('Price vs Odometer (Sample of 2000)')

#4. Average Price by Fuel Type
plt.subplot(2, 2, 4)
sns.barplot(data=df_cleaned, x='fuel', y='price')
plt.title('Average Price by Fuel Type')

plt.tight_layout()
plt.show()
```




```
#Preprocessing and Model Training

#1. Define features and target
X = df_cleaned.drop('price', axis=1)
y = df_cleaned['price']

#2. Identify categorical and numerical columns
numeric_features = ['year', 'odometer']
categorical_features = ['manufacturer', 'condition', 'cylinders', 'fuel', 'transmission', 'drive', 'type', 'paint_color']
```

```
#3. Create preprocessing pipelines
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

#4. Define the model pipeline
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=50, max_depth=10, random_state=42, n_jobs=-1))
])

#5. Train the model
print("Training the model... (this may take a minute)")
model.fit(X_train, y_train)

#6. Evaluation
y_pred = model.predict(X_test)
print(f"\n--- Model Performance ---")
print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"MAE: ${mean_absolute_error(y_test, y_pred):.2f}")
print(f"RMSE: ${np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")
```

```
Training the model... (this may take a minute)

--- Model Performance ---
R2 Score: 0.7162
MAE: $4718.88
RMSE: $7785.70
```

```
#7. Extract and plot Feature Importance
# Get feature names from the one-hot encoder
ohe_feature_names = model.named_steps['preprocessor'].named_transformers_['cat'].named_steps['onehot'].get_feature_names_out(cate
all_features = np.concatenate([numeric_features, ohe_feature_names])

#Get importance scores
importances = model.named_steps['regressor'].feature_importances_

#Create a dataframe for plotting
feature_importance_df = pd.DataFrame({'feature': all_features, 'importance': importances})
feature_importance_df = feature_importance_df.sort_values(by='importance', ascending=False).head(15)

plt.figure(figsize=(10, 6))
sns.barplot(data=feature_importance_df, x='importance', y='feature', palette='viridis')
plt.title('Top 15 Most Important Features for Price Prediction')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()
```